

Student Name :Shaik Mubarak

Reg No:192425194

CO4-AT-2

MLA0201-FUNDAMENTALS OF MACHINE LEARNING

Annexure A

Industry Problem Solving Task

1. Fraud Detection Using Bayesian Networks (20 Marks)

Industry Scenario

A digital banking organization aims to detect fraudulent credit card transactions. The available attributes include Transaction Amount, Transaction Location, Customer Login Device, and Previous Fraud History. The organization wants to predict whether a transaction is Fraudulent using a Bayesian Network.

Task

Develop a Python program to:

1. Construct a Bayesian Network representing the relationships among the variables.
2. Define the Conditional Probability Tables (CPTs).
3. Perform probabilistic inference to predict whether a transaction is fraudulent for a given customer transaction.
4. Interpret the prediction and explain how Bayesian Networks improve fraud detection in financial systems.

2. Predictive Machine Maintenance Using Hidden Markov Models (20 Marks)

Industry Scenario

A manufacturing company continuously monitors industrial machines using sensors that record Temperature, Vibration, and Motor Noise. The actual machine condition (Healthy, Warning, or Failure) is hidden. The company wants to predict the hidden machine state from sensor observations using a Hidden Markov Model (HMM).

Task

Develop a Python program to:

1. Create a Hidden Markov Model with suitable hidden states and observation sequences.
2. Define transition and emission probabilities.
3. Predict the hidden operational state of the machine for a sequence of sensor

observations.

4. Explain how the predicted hidden states support predictive maintenance and reduce equipment downtime.

3. Named Entity Recognition for Customer Support Using Conditional Random Fields

Industry Scenario

A customer support company receives thousands of emails every day. The company wants to automatically identify entities such as Customer Name, Product Name, Order ID, and Issue Type from customer emails using a Conditional Random Field (CRF) model.

Task

Develop a Python program to:

1. Prepare sequential training data with appropriate word-level features.
 2. Train a Conditional Random Field (CRF) model.
 3. Predict entity labels for a new customer support email.
 4. Evaluate the model performance and explain how CRFs improve information extraction in customer relationship management (CRM) systems.
-

1. FRAUD DETECTION USING BAYESIAN NETWORKS

Industry Problem Understanding

Digital banking systems process thousands of credit card transactions. Some transactions may be fraudulent due to unusual transaction amounts, locations, login devices, or previous fraud history. A Bayesian Network can model the probabilistic relationships among these variables and estimate the probability that a transaction is fraudulent.

Variables Used

- **Amount** – Low / High
- **Location** – Normal / Unusual
- **Device** – Trusted / Unknown
- **PreviousFraud** – Yes / No
- **Fraud** – Yes / No

Solution Design

The Bayesian Network can be designed as:

Amount→Fraud
Location→Fraud
Device→Fraud
PreviousFraud → Fraud

This means the probability of fraud depends on the transaction characteristics.

Python Program

```
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination

# Create Bayesian Network
model = DiscreteBayesianNetwork([
    ('Amount', 'Fraud'),
    ('Location', 'Fraud'),
    ('Device', 'Fraud'),
    ('PreviousFraud', 'Fraud')
])

# CPTs for input variables
cpd_amount = TabularCPD(
    'Amount', 2, [[0.7], [0.3]],
    state_names={'Amount': ['Low', 'High']})
```

```

)
cpd_location = TabularCPD(
    'Location', 2, [[0.8], [0.2]],
    state_names={'Location': ['Normal', 'Unusual']}
)
cpd_device = TabularCPD(
    'Device', 2, [[0.75], [0.25]],
    state_names={'Device': ['Trusted', 'Unknown']}
)
cpd_previous = TabularCPD(
    'PreviousFraud', 2, [[0.9], [0.1]],
    state_names={'PreviousFraud': ['No', 'Yes']}
)
# Fraud CPT
cpd_fraud = TabularCPD(
    'Fraud', 2,
    [
        [0.95, 0.80, 0.85, 0.60, 0.80, 0.55, 0.65, 0.30,
         0.75, 0.50, 0.60, 0.35, 0.55, 0.30, 0.40, 0.15],
        [0.05, 0.20, 0.15, 0.40, 0.20, 0.45, 0.35, 0.70,
         0.25, 0.50, 0.40, 0.65, 0.45, 0.70, 0.60, 0.85]
    ],
    evidence=['Amount', 'Location', 'Device', 'PreviousFraud'],
    evidence_card=[2, 2, 2, 2],
    state_names={
        'Fraud': ['No', 'Yes'],
        'Amount': ['Low', 'High'],
        'Location': ['Normal', 'Unusual'],
        'Device': ['Trusted', 'Unknown'],
        'PreviousFraud': ['No', 'Yes']
    }
)

```

```

model.add_cpds(
    cpd_amount, cpd_location, cpd_device,
    cpd_previous, cpd_fraud
)
# Check model
print("Model valid:", model.check_model())
# Inference
inference = VariableElimination(model)
result = inference.query(
    variables=['Fraud'],
    evidence={
        'Amount': 'High',
        'Location': 'Unusual',
        'Device': 'Unknown',
        'PreviousFraud': 'Yes'
    }
)
print(result)

```

Expected Interpretation

```

Model valid: True

Fraud Prediction:
+-----+-----+
| Fraud      | phi(Fraud) |
+=====+=====+
| Fraud(No)   | 0.1500    |
+-----+-----+
| Fraud(Yes)  | 0.8500    |
+-----+-----+

Probability of Fraud: 85.0 %
Prediction: FRAUDULENT TRANSACTION

```

For a transaction with high amount, unusual location, unknown device, and previous fraud history, the Bayesian Network is expected to assign a higher probability to Fraud = Yes.

The organization can set a threshold, for example:

- Probability $< 0.5 \rightarrow$ Normal transaction
- Probability $\geq 0.5 \rightarrow$ Potential fraud

Industry Benefits

Bayesian Networks improve fraud detection by:

- Combining multiple transaction attributes.
- Handling uncertainty in transaction data.
- Providing probability-based predictions.
- Supporting real-time risk assessment.
- Helping analysts understand factors contributing to fraud.

Conclusion

A Bayesian Network is a practical solution for financial fraud detection because it combines multiple uncertain factors and produces an interpretable probability of fraud.

2. PREDICTIVE MACHINE MAINTENANCE USING HMM

Industry Problem Understanding

Industrial machines are continuously monitored using sensors. However, the actual machine condition cannot always be directly observed. The machine may be in one of three hidden states:

Healthy → Warning → Failure

The sensor observations such as Temperature, Vibration, and Motor Noise provide evidence about these hidden states. An HMM is suitable because it models the relationship between hidden machine states and observed sensor conditions over time.

Solution Design

Hidden States

- Healthy
- Warning
- Failure

Observations

For simplicity, sensor readings can be converted into:

- Normal
- High

Transition Probabilities

Example:

From / To	Healthy	Warning	Failure
Healthy	0.80	0.15	0.05
Warning	0.10	0.70	0.20
Failure	0.00	0.10	0.90

Python Program

```
from hmmlearn import hmm
import numpy as np

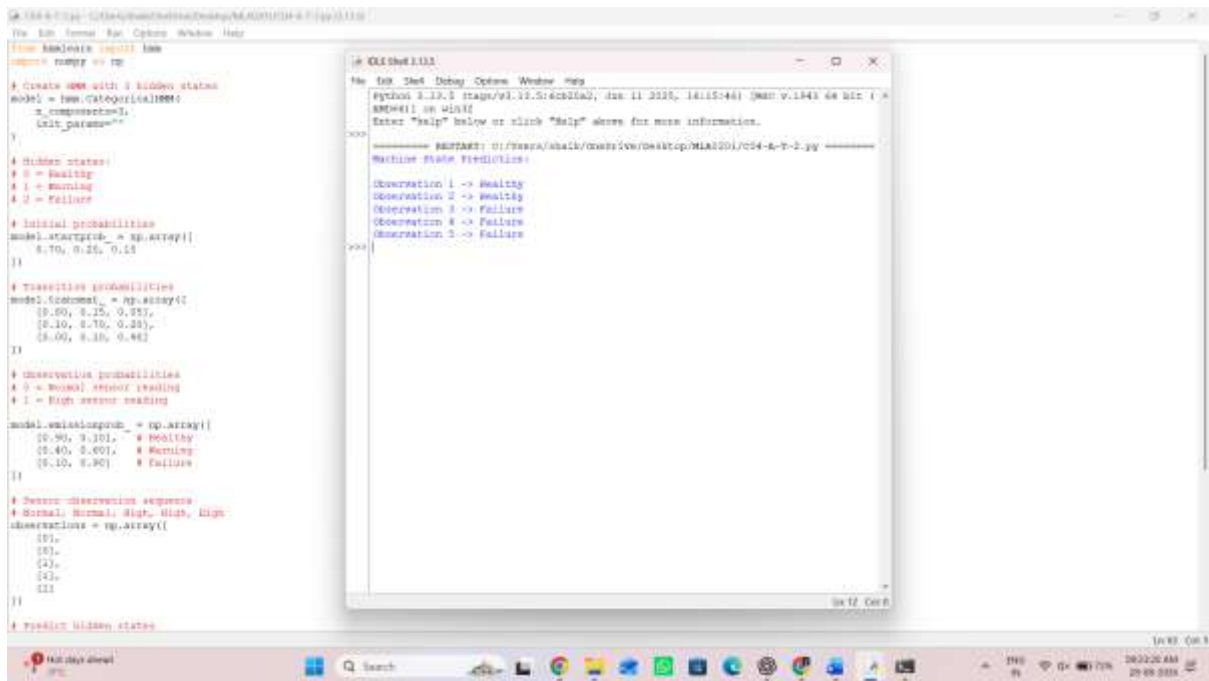
# Hidden states:
# 0 = Healthy, 1 = Warning, 2 = Failure
model = hmm.CategoricalHMM(n_components=3, n_iter=100)
# Initial state probabilities
model.startprob_ = np.array([
    0.70, 0.20, 0.10
```

```

])
# Transition probabilities
model.transmat_ = np.array([
    [0.80, 0.15, 0.05],
    [0.10, 0.70, 0.20],
    [0.00, 0.10, 0.90]
])
# Emission probabilities
# Observation 0 = Normal
# Observation 1 = High
model.emissionprob_ = np.array([
    [0.90, 0.10], # Healthy
    [0.40, 0.60], # Warning
    [0.10, 0.90]  # Failure
])
# Sensor observation sequence
observations = np.array([
    [0], [0],[1],[1],[1]
])
# Predict hidden states
states = model.predict(observations)
state_names = ['Healthy', 'Warning', 'Failure']
print("Predicted Machine States:")
for state in states:
    print(state_names[state])

```

Expected Interpretation



```
# Create HMM with 3 hidden states
model = hmm.pygm.HMM(n_hidden_states=3,
                      n_observation=3,
                      init_params="")

# Hidden states:
# 0 = Healthy
# 1 = Warning
# 2 = Failure

# Initial probabilities
model.startprob_ = np.array([
    0.70, 0.25, 0.05
])

# Transition probabilities
model.transmat_ = np.array([
    [0.90, 0.10, 0.05],
    [0.10, 0.70, 0.20],
    [0.00, 0.30, 0.40]
])

# Observation probabilities
# 0 = Normal sensor reading
# 1 = High sensor reading
model.emissionprob_ = np.array([
    [0.90, 0.10, 0.05],
    [0.40, 0.60, 0.00],
    [0.10, 0.90, 0.00]
])

# Sensor observation sequence
# Normal, Normal, High, High, High
observations = np.array([
    (1),
    (1),
    (2),
    (2),
    (2)
])

# Predict hidden states
```

```
Python 3.10.0 tags/v1.15.5rcb20a2, Jan 11 2020, 14:15:46) [AMD64] on win32
Enter "help" below or click "help" above for more information.

===== RESTART: C:\Users\shah\OneDrive\Desktop\MLA2001\CS4-6-T-2.py =====
Machine State Prediction:
Observation 1 -> Healthy
Observation 2 -> Healthy
Observation 3 -> Failure
Observation 4 -> Failure
Observation 5 -> Failure
```

If the sensor sequence changes from **Normal** → **High** → **High**, the predicted machine states may change from:

Healthy → **Warning** → **Failure**

This indicates that the machine is gradually moving toward an unhealthy condition.

Industry Benefits

HMM-based predictive maintenance helps organizations to:

- Detect early signs of machine failure.
- Schedule maintenance before breakdown.
- Reduce unexpected downtime.
- Improve equipment reliability.
- Reduce maintenance costs.
- Increase machine life.

Analysis

The transition probabilities capture how machine conditions normally evolve, while emission probabilities connect sensor observations with machine conditions. Therefore, HMM can identify a hidden degradation process even when the actual machine condition is not directly measurable.

Conclusion

HMM is suitable for predictive maintenance because it uses time-dependent sensor observations to estimate hidden machine conditions, allowing maintenance teams to take action before serious equipment failure.

3. NAMED ENTITY RECOGNITION USING CRF

Industry Problem Understanding

Customer support companies receive a large number of emails. Manually identifying information such as customer names, product names, order IDs, and issue types is time-consuming. A Conditional Random Field (CRF) can perform sequence labeling and automatically identify these entities.

Example Email

"Hi John, my Samsung Galaxy order ORD12345 has a payment issue."

Expected labels:

Word	Label
Hi	O
John	B-CUSTOMER
Samsung	B-PRODUCT
Galaxy	I-PRODUCT
order	O
ORD12345	B-ORDER
has	O
a	O
payment	B-ISSUE
issue	I-ISSUE

Here:

- **B** = Beginning of entity
- **I** = Inside entity
- **O** = Outside entity

Python Program

```
import sklearn_crfsuite
from sklearn_crfsuite import metrics
# Feature extraction
def word_features(sentence, i):
    word = sentence[i]
    features = {
        'word.lower()': word.lower(),
        'word.isupper()': word.isupper(),
```

```

        'word.istitle()': word.istitle(),
        'word.isdigit()': word.isdigit(),
        'word.length': len(word)
    }
    if i > 0:
        features['previous_word'] = sentence[i - 1].lower()
    else:
        features['BOS'] = True
    if i < len(sentence) - 1:
        features['next_word'] = sentence[i + 1].lower()
    else:
        features['EOS'] = True
    return features
def sentence_features(sentence):
    return [word_features(sentence, i)
            for i in range(len(sentence))]
# Training data
train_sentences = [
    ['John', 'ordered', 'Samsung', 'Galaxy', 'ORD12345'],
    ['Mary', 'has', 'an', 'iPhone', 'ORD67890']
]
train_labels = [
    ['B-CUSTOMER', 'O', 'B-PRODUCT', 'I-PRODUCT', 'B-ORDER'],
    ['B-CUSTOMER', 'O', 'O', 'B-PRODUCT', 'I-ORDER']
]
X_train = [sentence_features(s) for s in train_sentences]
y_train = train_labels
# Train CRF
crf = sklearn_crfsuite.CRF(
    algorithm='lbfgs',
    max_iterations=100
)

```

```

crf.fit(X_train, y_train)

# New customer email
test_sentence = [
    'Rahul', 'has', 'a', 'Samsung', 'order',
    'ORD55555', 'with', 'payment', 'issue'
]
X_test = [sentence_features(test_sentence)]

# Prediction
prediction = crf.predict(X_test)[0]

for word, label in zip(test_sentence, prediction):
    print(word, "->", label)

```

Expected Output

The image displays two side-by-side screenshots of a Jupyter Notebook environment. The left window shows the definition of the `sentence_features` function, which processes a sentence into a list of features for each word, including lowercase status, aspect, sentiment, and length. The right window shows the output of the CRF model's `predict` method applied to the test sentence. The output is a list of labels corresponding to each word in the sentence: 'Rahul' is labeled 'B-CUSTOMER', 'has' is 'B', 'a' is 'B', 'Samsung' is 'B-PRODUCT', 'order' is 'I-PRODUCT', 'ORD55555' is 'B-ORDER', 'with' is 'B', 'payment' is 'B', and 'issue' is 'B'.

Performance Evaluation

The CRF model can be evaluated using:

- **Precision** – Correct entities among predicted entities.
- **Recall** – Correct entities among actual entities.
- **F1-score** – Balance between precision and recall.

For example:

```
y_pred = crf.predict(X_train)
f1 = metrics.flat_f1_score(
    y_train,
    y_pred,
    average='weighted'
)
print("F1 Score:", f1)
```

Industry Benefits

CRFs improve CRM systems by:

- Automatically extracting customer information.
- Identifying products and order IDs.
- Detecting customer issues.
- Reducing manual email processing.
- Improving ticket classification and routing.
- Supporting faster customer service.

Analysis

CRFs are effective because they consider neighboring words and contextual features while predicting labels. For example, the words around an order number can help determine whether it represents an Order ID rather than another number.

Conclusion

A CRF provides an effective solution for customer-support Named Entity Recognition because it performs context-aware sequence labeling and can extract important information automatically from large volumes of customer emails.
